

How deep learning works

Artificial neural networks are, broadly speaking, inspired by the workings of the human brain's [neural circuits](#), whose functioning is driven by the complex transmission of chemical and electrical signals across distributed networks of nerve cells (neurons). In deep learning, the analogous “signals” are the weighted outputs of many nested mathematical operations, each performed by an artificial “neuron” (or *node*), that collectively comprise the neural network.

In short, a deep learning model can be understood as an intricate series of nested equations that maps an input to an output. Adjusting the relative influence of individual equations within that network using specialized machine learning processes can, in turn, alter the way the network maps inputs to outputs.

While that framework is very powerful and versatile, it comes at the expense of *interpretability*. There's often little, if any, intuitive explanation—beyond a raw mathematical one—for how the values of individual model parameters learned by a neural network reflect real-world characteristics of data. For that reason, deep learning models are often referred to as “black boxes,” especially when compared to traditional types of machine learning models informed by manual [feature engineering](#).

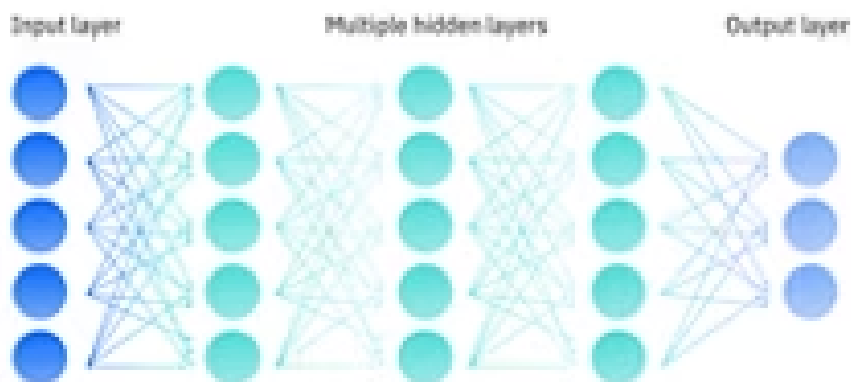
Relative to classic machine learning techniques, deep learning requires an exceedingly large amount of data and computational resources for training. Given the cost and complexity of the enterprise-level hardware needed to develop and implement sophisticated deep learning applications, cloud computing services have become an increasingly integral part of the deep learning ecosystem.

Deep neural network structure

Artificial neural networks comprise interconnected layers of artificial “neurons” (or *nodes*), each of which performs its own mathematical operation (called an “[activation function](#)”). There exist many different activation functions; a neural network will often incorporate multiple activation functions within its structure, but typically all of the neurons in a given layer of the network will be set to perform the same activation function. In most neural networks, each neuron in the *input layer* is connected to each of the neurons in the following layer, which are themselves each connected to the neurons in layer after that, and so on.

The output of each node's activation function contributes part of the input provided to each of the nodes of the following layer. Crucially, the activation functions performed at each node are *nonlinear*, enabling neural networks to model complex patterns and dependencies. It's the use of nonlinear activation functions that distinguishes a deep neural network from a (very complex) linear regression model.

While some specialized neural network architectures, such as [mixture of expert](#) models or [convolutional neural networks](#), entail variations, additions or exceptions to this arrangement, all neural networks employ some version of this core structure. The specific number of layers, number of nodes within each layer, and the activation functions chosen for each layer's nodes are [hyperparameters](#) to be determined manually prior to training.



A standard feedforward neural network with 3 hidden layers.

Each of these myriad neuron-to-neuron connections is multiplied by a unique *weight*, which amplifies (or diminishes) the influence of each connection. The input provided to each neuron's activation function can be understood as the *weighted sum* of the outputs of each neuron in the previous layer. There's usually also a unique *bias* term added to each activation function, which functions similarly to the bias term of a common regression function.

During training, the neural network “learns” through adjustments to each of these weights and bias terms that yield more accurate outputs. These are the [model's parameters](#): when you read about, for instance, a [large language model \(LLM\)](#) having 8 billion “parameters,” that number reflects every single weighted neuron-to-neuron connection and neuron-specific bias in the model's neural network.

The intermediate layers, called the network's *hidden layers*, are where most of the learning occurs. It's the inclusion of *multiple* hidden layers that distinguishes a deep learning model from a “non-deep” neural network, such as a [restricted Boltzmann machine \(RBN\)](#) or standard multilayer perceptron (MLP). The presence of multiple hidden layers allows a deep learning model to learn complex hierarchical features of data, with earlier layers identifying broader patterns and deeper layers identifying more granular patterns.

To perform [inference](#), the network completes a *forward pass*: the input layer receive input data, usually in the form of a [vector embedding](#), with each input neuron processing an individual feature of the input vector. For example, a model that works with 10x10 pixel grayscale images will typically have 100 neurons in its input layer, with each input neuron corresponding to an individual pixel. Neural networks thus typically require input vectors to be fixed to a certain size, though preprocessing techniques like pooling or normalization can provide some flexibility with regard to the size of the original input data itself.

The data is progressively transformed and passed along to the nodes of each subsequent layer until the final layer. The activation functions of the neurons in the output layer compute the network's final output prediction. For instance, each output node of a deep [classification model](#) might perform a [softmax function](#) that essentially takes a numerical input and scales it to a probability, between 0–1, that the input belong a potential classification category. The model would then output the category corresponding to whichever output node yielded the highest output.

Training deep neural networks

While the theoretical potential of deep neural networks was always apparent, it was not initially known how to efficiently train them. The goal of optimizing model parameters through training is to reduce the error of the network's final outputs—but separately isolating and calculating how *each* of a neural network's thousands, if not millions or billions, of interconnected weights contributed to overall error is thoroughly impractical.

This obstacle was overcome by the introduction of two essential algorithms: [backpropagation](#) and [gradient descent](#).

Backpropagation

[Backpropagation](#), short for “backward propagation of error,” is an elegant method to calculate how changes to any individual weight or bias in a neural network will affect the accuracy of model predictions.

Recall that an artificial neural network is essentially a series of nested mathematical functions: the outputs of one layer's neurons serve as the inputs to the next layer's neurons, and so on. During training, those interconnected equations are nested into yet another function: a [loss function](#) that measures the average difference (or “loss”) between the desired output (or “[ground truth](#)”) for a given input and the neural network's actual output for each forward pass.

Once the model's initial hyperparameters have been determined, training typically begins with a random initialization of model parameters. The model makes predictions on a batch of examples from the training dataset and the loss function tracks the error of each prediction. The goal of

training is to iteratively optimize parameters until average loss has been reduced to below some acceptable threshold.

Backpropagation entails a single end-to-end *backwards pass* through the network, beginning with the output of the loss function and working all the way back to the input layer. Using the [chain rule of calculus](#), backpropagation calculates the “gradient” of the loss function: a vector of partial derivatives of the loss function with respect to each variable in every equation that ultimately nests into the calculation of the loss function. In other words, it describes how increasing or decreasing the output of any individual neuron’s activation function will affect overall loss—which, by extension, describes how any *changes to the any of the weights those outputs are multiplied by* (or to the bias terms added to those outputs) will increase or decrease loss.

Gradient descent

The gradient computed during backpropagation then serves as an input to a [gradient descent](#) algorithm.

Moving down—*descending*—the gradient of the loss function will decrease loss (and thereby increase accuracy). Since the gradient calculated during backpropagation contains the partial derivatives of the loss function with respect to every model parameter, we know which direction to “step” the value of each of parameter to reduce loss.

Each step entails an update of the model’s parameters, and reflects the model “learning” from its training data. Our goal is to iteratively update weights until we have reached the minimum gradient. The object of gradient descent algorithms is to find the specific parameter adjustments that will “descend” the gradient most efficiently.

Implementing deep learning models

There are a number of open source frameworks for developing deep learning models, whether training a model from scratch or [fine-tuning](#) a [pretrained model](#). These [machine learning libraries](#) offer a variety of preconfigured modules and workflows for building, training and evaluating neural networks, simplifying and streamlining the process of development process

Among the most popular open source frameworks for working with deep learning algorithms are [PyTorch](#), TensorFlow and (particularly for LLMs) the Hugging Face Transformers library. It’s recommended to learn [Python](#) prior to working with these frameworks.

Types of deep learning models

Despite their inherent power and potential, adequate performance on certain tasks remains either impossible or impractical for conventional (“vanilla”) deep neural networks. Recent decades have seen several innovations to the standard neural network architecture, each aimed at enhanced performance on particular tasks and types of data.

It's worth noting that a given type of neural network might lend itself to multiple types of deep learning *models*, and vice versa. For instance, an autoencoder model used for image tasks might leverage a convolutional neural network-based architecture; diffusion models can utilize CNN-based or transformer-based architectures.

Convolutional neural networks (CNNs)

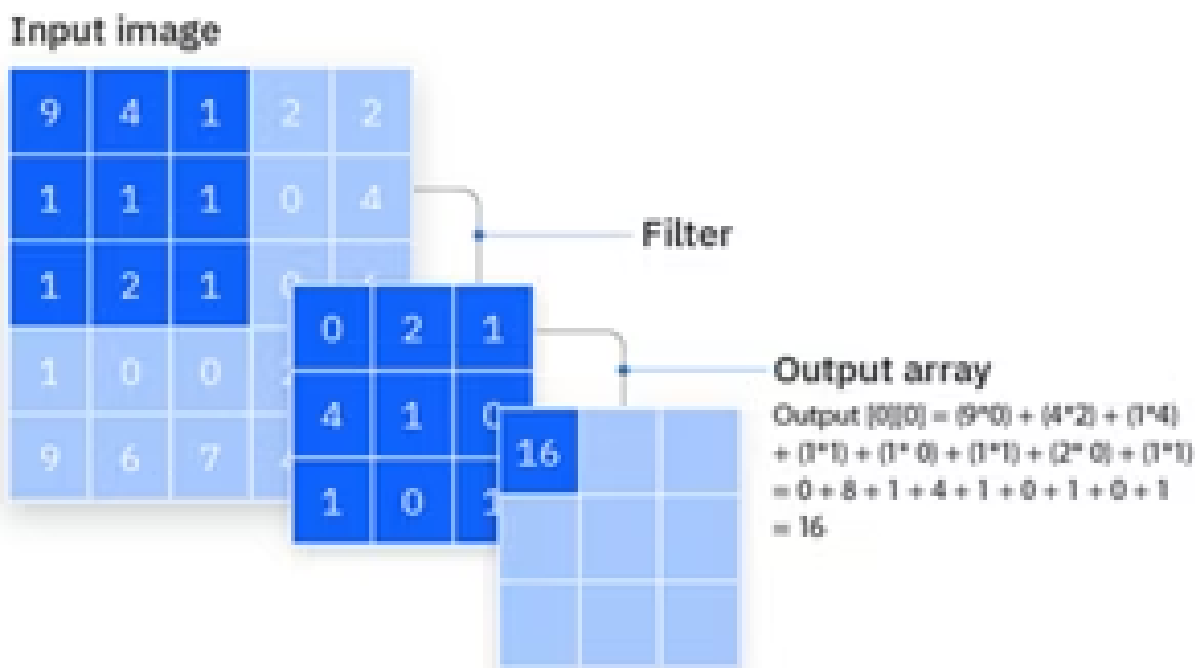
Convolutional neural networks (CNNs) are primarily (but not exclusively) associated with computer vision tasks such as object detection, image recognition, image classification and image segmentation, as they excel at “local” pattern recognition (such as relationships between adjacent pixels in an image).

The intuition behind the development of CNNs was that for certain tasks and data modalities—like classifying high-resolution images with hundreds or thousands of pixels—sufficiently sized neural networks comprising only standard, fully connected layers would have far too many parameters to generalize well to new data post-training. In other words, they'd be computationally inefficient and prone to overfitting training data rather than learning genuinely useful real-world patterns.

In theory, a neural network that can detect distinct shapes and other meaningful features could save computational power by extracting said features from the raw image for further processing (and discarding information about regions of the image without meaningful features). One way to do so would be to use *filters*: small, 2-dimensional arrays of numbers whose values correspond to the shape of useful features. For instance, the values of a filter that scans an image's pixels for top left corners might look like this:

$$\begin{bmatrix} 10 & 10 & 10 & 10 & 10 \\ 10 & 0 & 0 & 0 & 0 \\ 10 & 0 & 0 & 0 & 0 \\ 10 & 0 & 0 & 0 & 0 \\ 10 & 0 & 0 & 0 & 0 \end{bmatrix}$$

Now imagine that 5x5 filter being multiplied by a 5x5 grid of pixels in an input image. In mathematical parlance, this is called a *convolution*: a mathematical operation where one function modifies (or *convolves*) a second function. If the pixel values resemble those of the filter, the product of that multiplication (the *dot product*) will be large and the feature those pixels represent will be captured; if not, the dot product will be small and the pixels ignored.



A small section of an image's pixel values (left) are multiplied by a convolution filter (middle), yielding a lower dimensional representation of the original pixels (right) that reflects how the original pixels resemble the information represented by the filter.

CNNs add *convolution layers*, containing far fewer nodes than standard fully connected layers that act as such filters. Rather than requiring a unique node (with a unique weight) corresponding to each individual pixel in the image, a convolution layer's filter strides along the entire image, processing one correspondingly-sized grid of pixels at a time. This not only extracts useful information, but also significantly reduces the number of unique model parameters required to process the whole image.

CNNs are typically much "deeper" (in terms of number of layers) than standard neural networks, but, because convolution layers contain relative few neurons, still efficient in terms of total parameter count. As data traverses the CNN, each convolutional layer extracts progressively more granular features, assembling a "feature map." The final feature map is eventually passed to a standard fully connected layer that performs final predictions. In training, the model naturally learns weights for the convolution layers that result in their filters capturing features conducive to accurate final predictions.

Recurrent neural networks (RNNs)

[Recurrent neural networks \(RNNs\)](#) are used for tasks involving sequential data, such as time-series forecasting, [speech recognition](#) or [natural language processing \(NLP\)](#).

Whereas conventional feedforward neural networks map a single input to a single output, RNNs map a *sequence* of inputs to an output by operating in a recurrent loop in which the output for a

given step in the input sequence serves as input to the computation for the following step. In effect this creates an internal “memory” of past inputs, called the *hidden state*. Updated after each time step, this hidden state allows an RNN to maintain an understanding of context and order.

While the notion of a single “rolled up” layer is useful for understanding the concept, this recurrence can also be understood as data traversing a series of multiple layers that share identical weights.



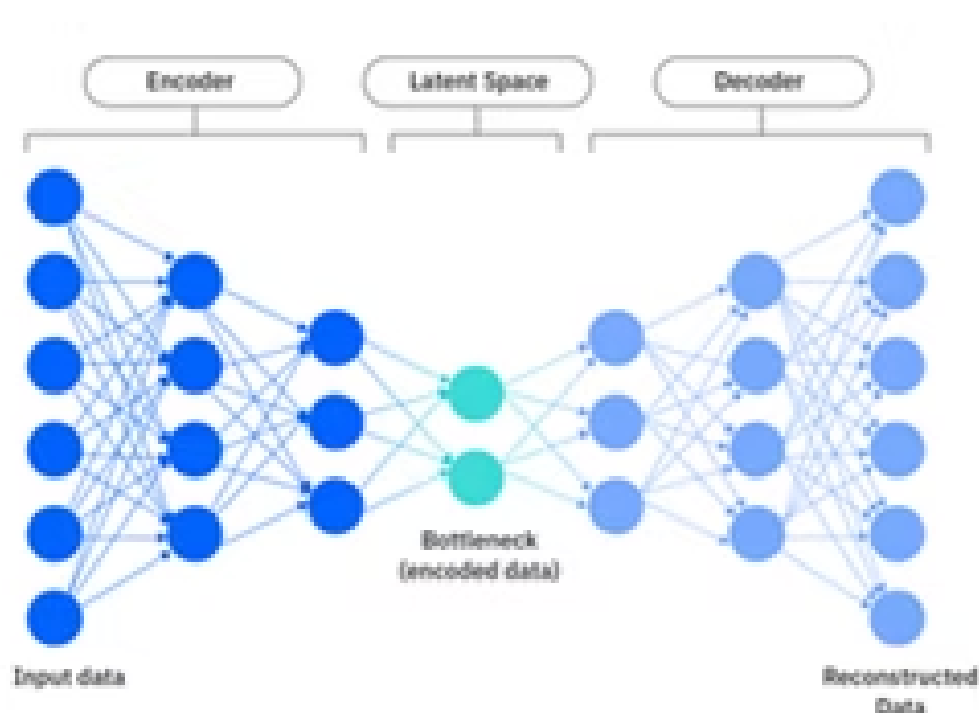
An RNN, displayed in "rolled" and "unrolled" forms

This leads to some fundamental shortcomings of conventional RNNs, particularly with regard to training. Recall that backpropagation calculates the gradient of the loss function, which determines how each individual model parameter should be either increased or decreased. When each of these parameter updates is repeated across too many “identical” recurrent layers, these updates scale exponentially: enlarging parameters can lead to *exploding gradient*, and minimizing parameters can lead to *vanishing gradients*. Both issues can introduce instability in training, slow training or even altogether halt training. Standard RNNs are thus limited to processing relatively short sequences.

Various enhancements to the basic RNN architecture, such as long short-term memory (LSTM) networks or gated recurrent units (GRUs), mitigate these problems and increase the model’s ability to model long-range dependencies.

Autoencoders

[Autoencoders](#) are designed to compress (or *encode*) input data, then reconstruct (decode) the original input using this compressed representation. In training, they're optimized to minimize *reconstruction loss*: the divergence between the reconstructed data point and the original input data. Though this type of deep learning uses unlabeled, unstructured data, autoencoders are generally considered to be an archetypal example of [self-supervised learning](#).



In essence, this forces the model to learn weights that result in the compressed representation retaining only the most essential, meaningful subset of the input data's features. In machine learning parlance, autoencoders model the *latent space*.

Autoencoders have a variety of use cases, such as data compression, [dimensionality reduction](#), [feature extraction](#), denoising corrupted data, and fraud detection.

In most cases, the decoder network serves only to help train the encoder and is discarded after training. In [variational autoencoders \(VAEs\)](#), a type of [generative model](#), the decoder is retained and used to generate new data points by adding some random noise to the latent representations learned by the encoder before reconstruction.

Transformer models

The advent of [transformer models](#), first introduced in a seminal 2017 paper from Google DeepMind titled "[Attention is all you need](#)", was a watershed moment in deep learning that led directly to the current era of generative AI.

Like RNNs, transformers are inherently designed to work with sequential data. The defining feature of transformer models is their unique [self-attention mechanism](#), from which transformers derive their impressive ability to discern the relationships (or dependencies) between each part of an input sequence. More importantly, this attention mechanism enables transformers to selectively focus on (or “attend to”) the parts of an input sequence that are most relevant at any given moment.

Attention mechanisms were first introduced in the contexts of RNNs used for [machine translation](#). But unlike RNNs, transformers don’t use recurrent layers; a standard transformer architecture uses only attention layers and standard feedforward layers, leveraging a novel structure inspired by the logic of [relational databases](#).

Transformers are most commonly associated with large language models (LLMs) and, by association, NLP use cases such as text generation, [chatbots](#) and [sentiment analysis](#). But they’re extremely versatile models capable of processing any sequential data modality, including audio or [time series](#) data. Even data modalities like image data can be processed by vision transformers (ViTs) through clever workarounds to represent them as a sequence.

Though transformer models have yielded state-of-the-art results across nearly every domain of deep learning, they are not necessarily the optimal choice for any and all use cases. For instance, while ViTs have achieved top performance ranks across benchmarks for computer vision tasks, CNNs are significantly faster and more computationally efficient. For tasks like object detection or image segmentation, the choice between a transformer or CNN often comes down to whether a given deep learning application must prioritize maximum accuracy or real-time feedback.

Mamba models

First introduced in 2023, [Mamba models](#) are a novel deep learning architecture for sequential data. Derived from a variation of [state space models \(SSMs\)](#), Mamba has interesting theoretical connections to RNNs, CNNs and transformer models. Most importantly, Mamba shares with transformers the ability to selectively prioritize (or discard) past information based on its relevance at a given moment—albeit with an entirely unique mechanism for doing so.

To date, Mamba is perhaps the only architecture to meaningfully rival transformers in the domain of LLMs, offering comparable performance with significantly greater computational efficiency due to its much less memory-intensive algorithm.

Generative adversarial networks (GANs)

Like VAEs, [generative adversarial networks \(GANs\)](#) are neural networks used to create new data resembling the original training data. GANs are a joint architecture combining two deep learning

networks trained *adversarially* in a zero-sum game.

The **generator** network creates new data points, such as original images. Any generative architecture capable of producing the desired output can be used for a GANs generator network. Its sole defining characteristic is how it interacts with the discriminator, and its sole requirement is that algorithm be differentiable (and thus able to optimized through backpropagation and gradient descent).

The **discriminator** is provided both “real” images from the training dataset and “fake” images output by the generator and tasked with determining of a given image is real or fake. Like the generator, the discriminator can take the form of any suitable architecture.

First, the discriminator is trained to correctly classify fake images. During that time, the generator’s weights are frozen.

Next, the weights of the discriminator are frozen and feedback from the discriminator is used to train the generator. The generator’s weights are optimized to yield images more likely to fool the discriminator.

The process is repeated: the discriminator receives another assortment of “real” images from training data and “fake” images from the generator—which are now, presumably, more convincing. The discriminator once again predicts whether each image is real or fake and is once again updated.

Once again, feedback from the (presumably harder-to-fool) discriminator is used to further train the generator.

The process continues iteratively until the discriminator is no longer able to discern between real and fake samples.

GANs are capable of learning to produce incredibly accurate examples, but the adversarial nature of the process makes training inherently tricky and unstable.

Diffusion models

[Diffusion models](#) are among the most prominent neural network architectures in [generative AI](#). They’re both practical and performant, offer the training stability of VAEs and the output fidelity of GANs. They’re most commonly used for image generation, but are also capable of generating text, video and audio data.

Like autoencoders, diffusion models are essentially trained to destruct an image and then accurately reconstruct it, albeit in an entirely different manner. In training, diffusion models learn to

gradually *diffuse* a data point step-by-step with Gaussian noise, then *reverse* that process to reconstruct the original input. In doing so, they gain the ability to generate new samples (resembling the original training data) by “denoising” a sample of random noise.

Latent diffusion models are essentially a hybrid of VAEs and diffusion models: they first compress (encode) input data down to latent space, then perform the diffusion process, and then feed the result to a decoder that upsamples it to the desired image size.

While diffusion models typically use a CNN-based architecture—specifically, the *U-net* architecture used prominently for segmentation in medical imaging—some leverage a transformer-based architecture instead.

Graph neural networks

[Graph neural networks \(GNNs\)](#) are designed for tasks that require modeling more complex relationships between different entities than are typical of most data modalities.

Consider image data, in which the pixels of an image are arranged in a 2-dimensional grid: any one pixel is directly connected to, at most, 8 adjacent pixels. A standard CNN is well-suited to modeling such relationships. But that capability extends poorly to modeling the relationships within, for example, a social media network in which a given user may be connected directly to thousands of other users and indirectly to many thousands more.

The structure of graph neural networks allows for more complex and irregular representations of data than are possible in the unidirectional flow of data inherent to other neural network architectures.

Think Keynotes